

Chapter 1

Introduction

What are Embedded Systems?

- Computers whose job is not primarily information processing, but rather is interacting with physical processes.
- Examples:
 - Automotive controllers
 - Avionics (航空)
 - Medical devices
 - Industrial control
 - Energy management and conservation
- A broader view is that of *cyber-physical systems (CPS)*
(信息物理融合系统)

What are Embedded Systems?

- Computational systems
not first-and-foremost a computer
- Integral with physical processes
sensors (传感器), actuators (激励器), physical dynamics
- Reactive
at the speed of the environment (timing matters!)
- Heterogeneous (异构)
hardware/software/network, mixed architectures
- Networked
Concurrent, distributed, dynamic

Cyber-Physical Systems (CPS):

- *Orchestrating networked computational resources with physical systems*
- 3C: Computing, Control and Communication

From Desktops to Cyber-Physical Systems

- ❑ Traditional computers: Stand-alone device running software applications (e.g. data processing)
- ❑ Traditional controllers: Devices interacting with physical world via sensors and actuators (e.g. thermostat (温控器))
- ❑ Embedded Systems
 - Special-purpose system with integrated microcontroller/software
 - Cameras, watches, washing machines...
- ❑ Cyber-physical systems
 - Computing devices communicating with one another and interacting with the physical world via sensors/actuators
 - Computing + Control + Communication
 - Example: Team of autonomous robots

Foundations of Cyber-Physical Systems

❑ CPS in industry

- Technologies for low-cost devices available, but...
- Challenge: Managing complexity to produce reliable systems

❑ Foundations for traditional computing

- Limits of computation: Turing machines (图灵机) ...
- Efficiency of problem solving: Algorithms and complexity
- Correctness: Formal specification and verification
- Complexity of software: Programming abstractions

❑ Does design of CPS need new theoretical foundations?

- What are the distinguishing features of CPS?

Functional vs Reactive Computation

❑ Classical model of computation: Functional

- Given inputs, a program produces outputs
- Desired functionality described by a mathematical function
- Example: Sorting of names, Shortest paths in a weighted graph
- Theory of computation provides foundation for this view

❑ Reactive （反应式系统）

- System interacts with its environment via inputs and outputs in an ongoing manner
- Desired behaviors: which sequences of observed input/output interactions are acceptable?
- Example: controller in a car

❑ CPS: Reactive

Sequential vs Concurrent Computation

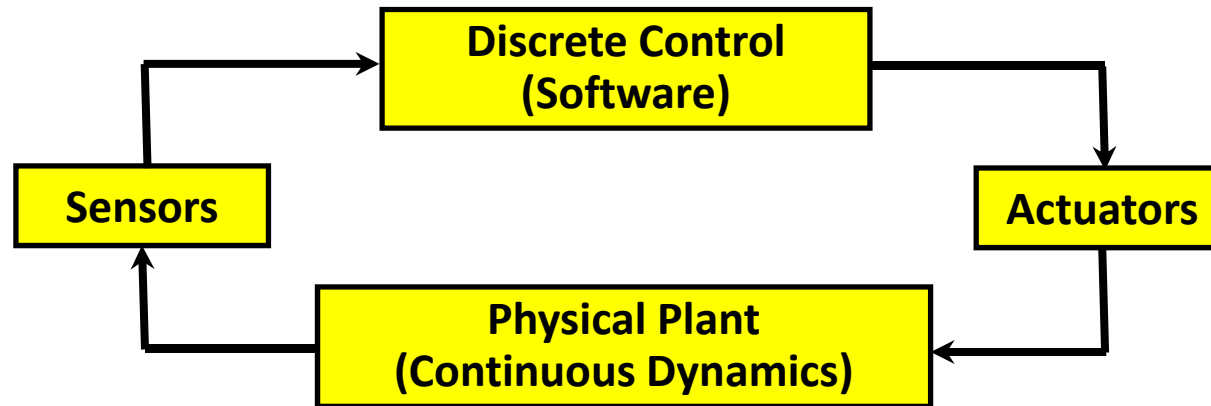
❑ Classical model of computation: Sequential

- A computation is a sequence of instructions executed one at a time
- Well understood and canonical model: Turing machines

❑ Concurrent Computation

- Multiple components/processes exchanging information and evolving concurrently
- Logical vs physical concurrency
- Broad range of formal models for concurrent computation
- Key distinction: Synchronous vs Asynchronous

Feedback Control of the Physical World



❑ Classical control theory: Continuous-time systems

- Key ingredient: Modeling of dynamical systems by differential equations (微分方程)
- Theory of linear systems (线性) for design and analysis

❑ Hybrid systems: Discrete + continuous dynamics

- Multiple modes of operation
- Discrete software

Real-Time Computation

- ❑ Traditional computing infrastructure (programming languages, operating systems, and processor architecture) do not support real time explicitly
 - Efficient, flexible and portable implementations
 - Suitable for general purpose computing tasks, but not for CPS
- ❑ Real-Time
 - Timing of events critical to correctness
 - Need a way to predict and model delays at design time
 - Key: Predictable and timely performance
 - Need to worry about resource contention and allocation

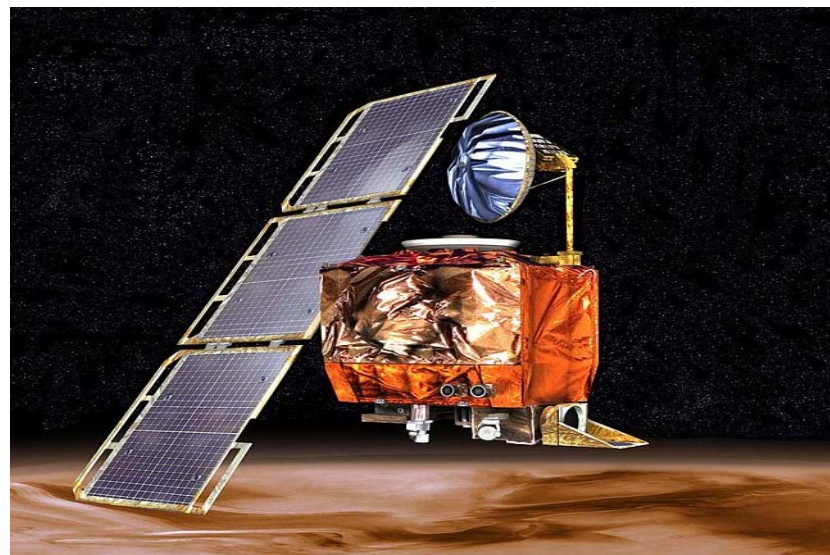
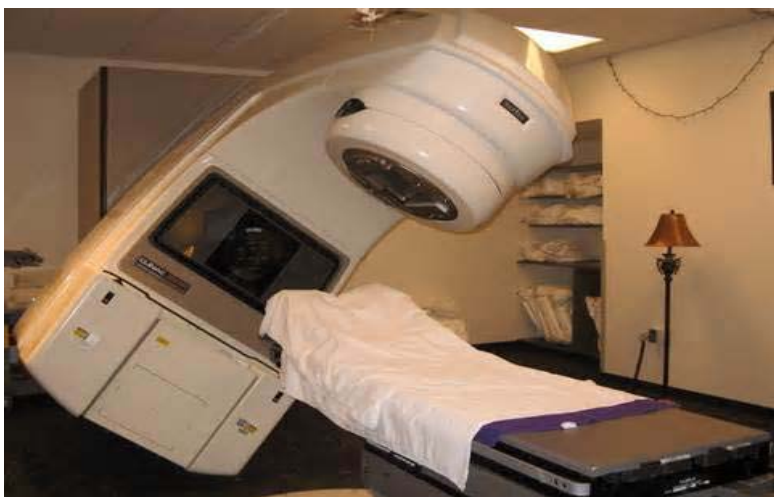
Safety-Critical (安全攸关) Applications

- ❑ Traditional verification and validation approach
 - Design, implementation, followed by extensive testing
 - Software bugs are common-place
 - Fixing of bugs can be expensive
- ❑ Safety-critical applications
 - Bugs lead to catastrophic (灾难性的) failures
 - Establishing correctness at design time a high priority
- ❑ Formal methods
 - Formalize correctness requirements in a precise manner
 - Develop a model of system and its environment at design time
 - Establish that the system meets the requirements

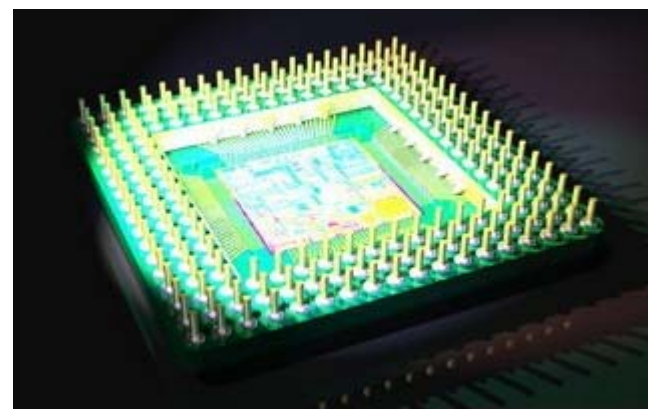


1996年阿丽亚娜火箭爆炸
损失超过**3.7亿**美元

1985-1987年间，Therac-25软件漏洞造成至少**5人**死亡



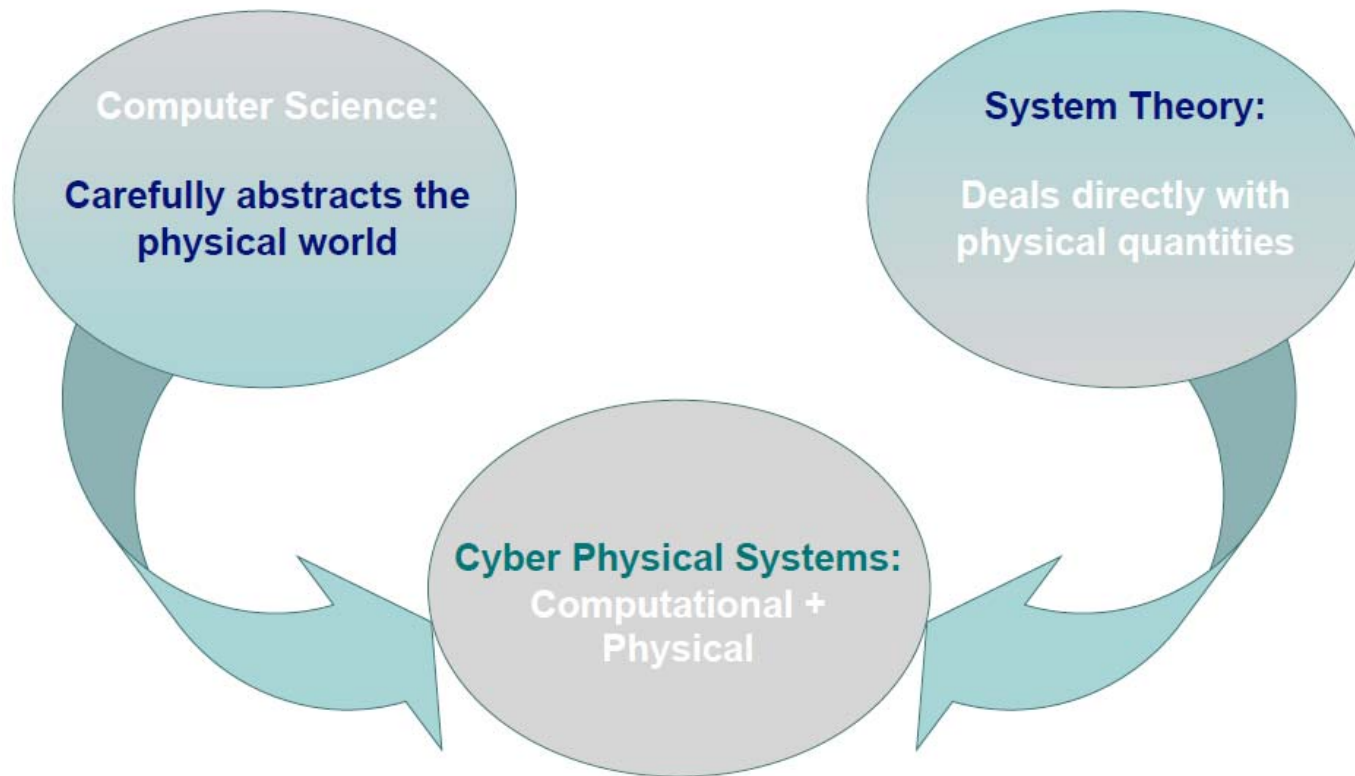
1999年火星气候探测器坠毁，
损失**1.25亿**美元



英特尔奔腾芯片出错，损失**4.75亿**美元及**公司信誉**

FAULT

CPS Multidisciplinary (多领域)



Content of an Embedded Systems Course

Traditional focus

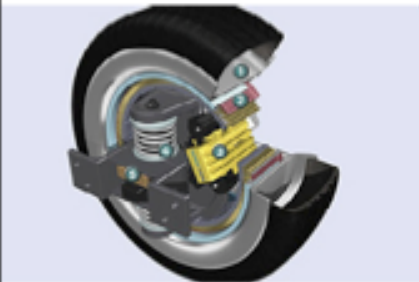
- Hardware interfacing
- Interrupts
- Memory systems
- C programming
- Assembly language
- FPGA design
- RTOS design
- ...

CPS focus

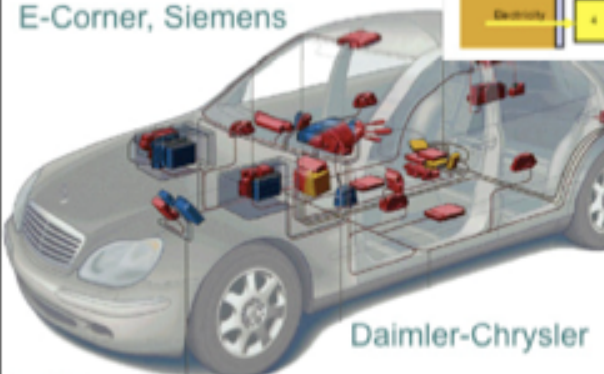
- Modeling
- Timing
- Dynamics
- Imperative logic
- Concurrency
- Verification
- ...

Cyber-Physical Systems (CPS): *Orchestrating networked computational resources with physical systems*

Automotive

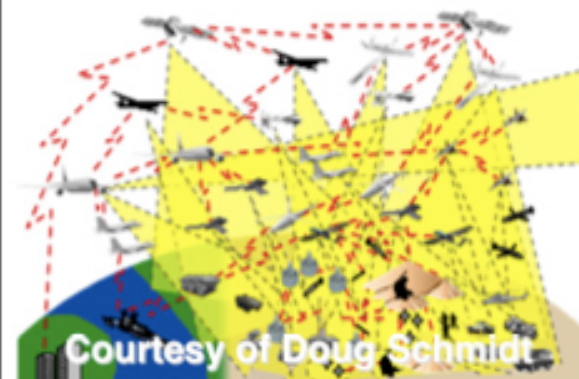


E-Corner, Siemens



Daimler-Chrysler

Military systems:

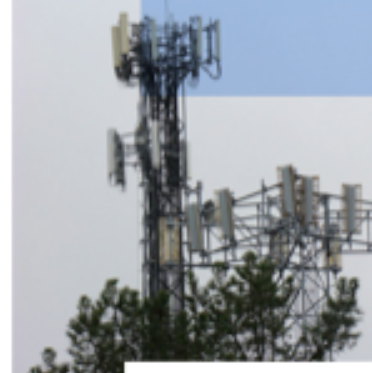


Courtesy of Doug Schmidt

Building Systems



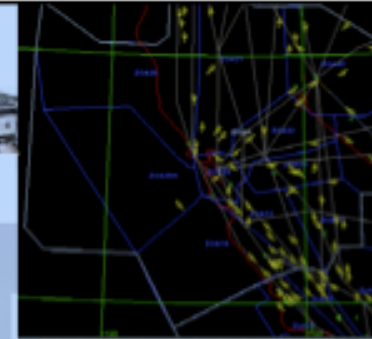
Telecommunications



Avionics



Transportation
(Air traffic
control at
SFO)



Instrumentation
(Soleil Synchrotron)



Factory automation



Courtesy of Kuka Robotics Corp.

Power generation and distribution



Courtesy of
General Electric

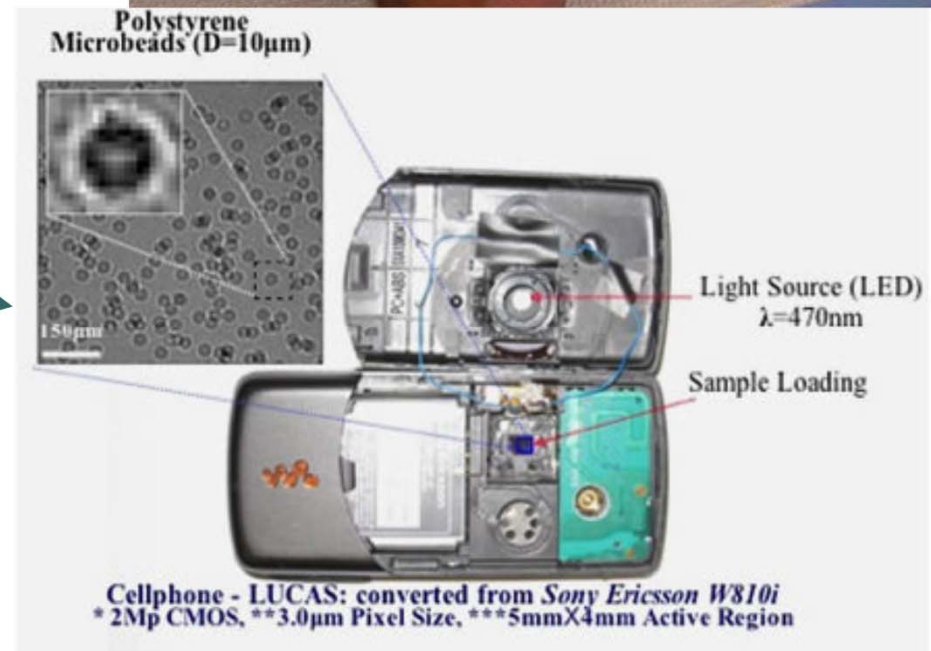
Example: Medical Devices

Emerging direction: Cell phone based medical devices for affordable healthcare

e.g. “Telemicroscopy” project at Berkeley



e.g. Cell-phone based blood testing device developed at UCLA



CPS Example – Printing Press

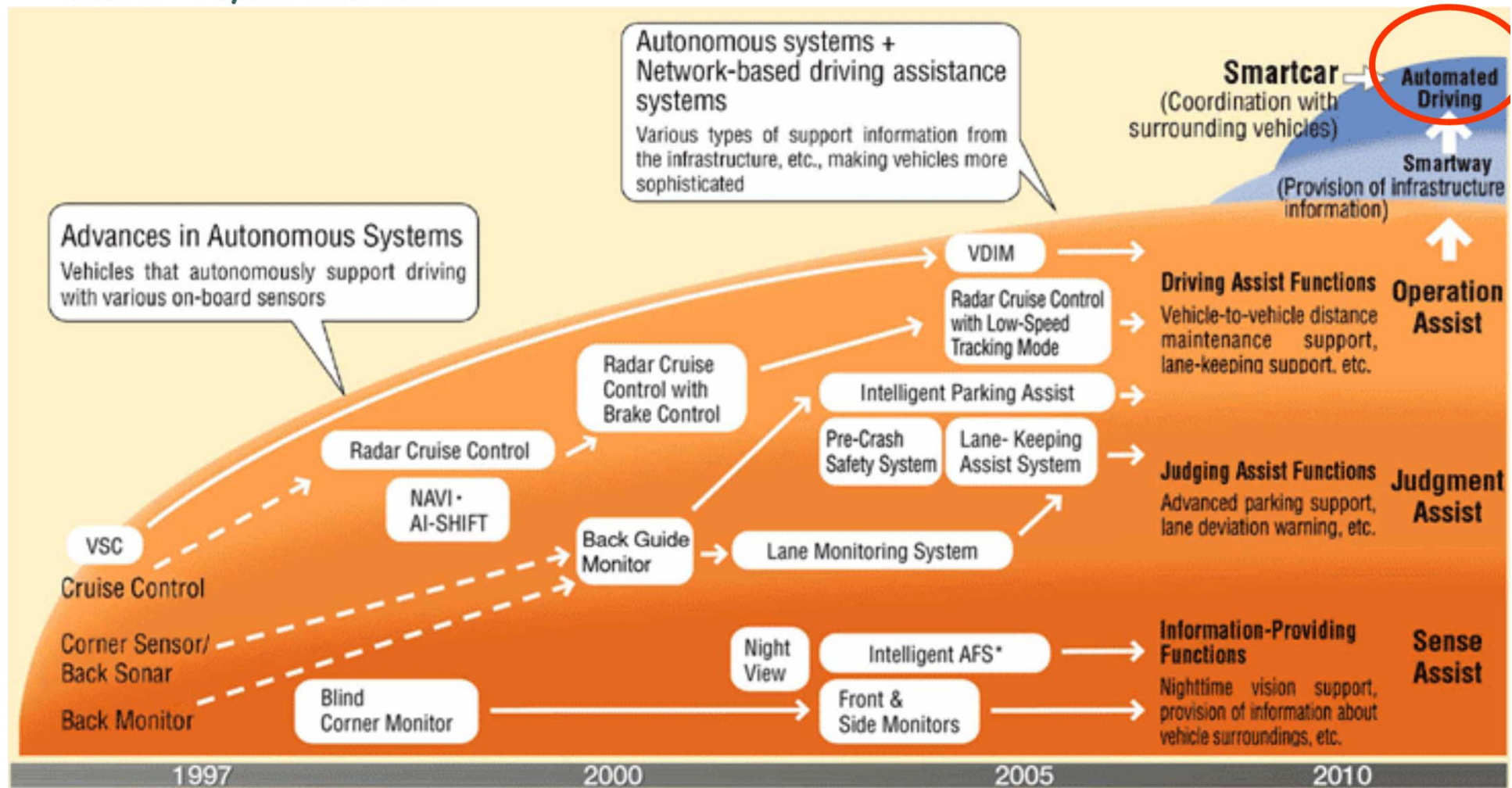


Bosch-Rexroth

- High-speed, high precision
 - Speed: 1 inch/ms
 - Precision: 0.01 inch
 - > Time accuracy: 10us
- Open standards (Ethernet)
 - Synchronous, Time-Triggered
 - IEEE 1588 time-sync protocol
- Application aspects
 - local (control)
 - distributed (coordination)
 - global (modes)

Example: Toyota autonomous vehicle technology roadmap, c. 2007

Source: Toyota Web site



Example: Automotive electronics today

About 80 computers (*electronic control units*, 电子控制单元, ECUs) in a premium car today:

- engine control, transmission, anti-lock brakes(防抱死制动), electronic suspension (电控悬架), parking assistance, climate control, audio system, “body electronics” (seat belt, etc.), display and instrument panel, etc.
- linked together by CAN bus, FlexRay with up to 2km of wiring.
- growing fraction of development costs, manufacturing costs, and fuel consumption.

A Key Challenge on the Cyber Side: Real-Time Software

- *Correct execution of a program in C, C#, Java, Haskell, etc. has nothing to do with how long it takes to do anything. All our computation and networking abstractions are built on this premise.*
- Timing of programs is not repeatable, except at very coarse granularity.
- Programmers have to step outside the programming abstractions to specify timing behavior.

Techniques Exploiting the Fact that Time is Irrelevant

- Programming languages
- Virtual memory
- Caches
- Dynamic dispatch
- Speculative execution
- Power management (voltage scaling)
- Memory management (garbage collection)
- Just-in-time (JIT) compilation
- Multitasking (threads and processes)
- Component technologies (OO design)
- Networking (TCP)
- ...

First Challenge

- Models for the physical world and for computation diverge(发散).
 - physical: time continuum, ODEs (常微分方程), dynamics
 - computational: a “procedural epistemology (认识论),” logic
- There is a huge cultural gap.
 - Physical system models must be viewed as semantic frameworks,
 - and theories of computation must be viewed as alternative ways of talking about dynamics.

Second Challenge

- We typically learn to *use* modeling techniques, not to *evaluate* modeling techniques.
- “this is how computers work”
- “this equation describes that feedback circuit (反馈电路)”
- rather than
 - “this is how Von Neumann proposed that we control automatic machines”
 - “ignoring the intrinsic randomness and latency (延迟) in this circuit, Black proposed that we could idealize its behavior in this way”
- We need to think about meta-modeling, not just modeling. They must learn to think critically about modeling methods, not just about models.

What is Modeling?

- Developing insight about a system, process, or artifact through imitation (模仿) .
- *A model is the artifact that imitates the system, process, or artifact of interest.*
- *A mathematical model is model in the form of a set of definitions and mathematical formulas.*

What is Model-Based Design?

- Create a mathematical model of all the parts of the embedded system
 - Physical world
 - Control system
 - Software environment
 - Hardware platform
 - Network
- Construct the implementation from the model
 - Construction may be automated, like a compiler
 - More commonly, portions are automatically constructed

Modeling Techniques in this Course

- Models that are abstractions of system dynamics (how things change over time)
- Examples:
 - Feedback control systems – time-domain modeling
 - Modeling modal behavior – FSMs, hybrid automata
 - Modeling software – concurrency, real-time models
 - Modeling networks – latencies, error rates, packet loss

Other Modeling Techniques we will talk about

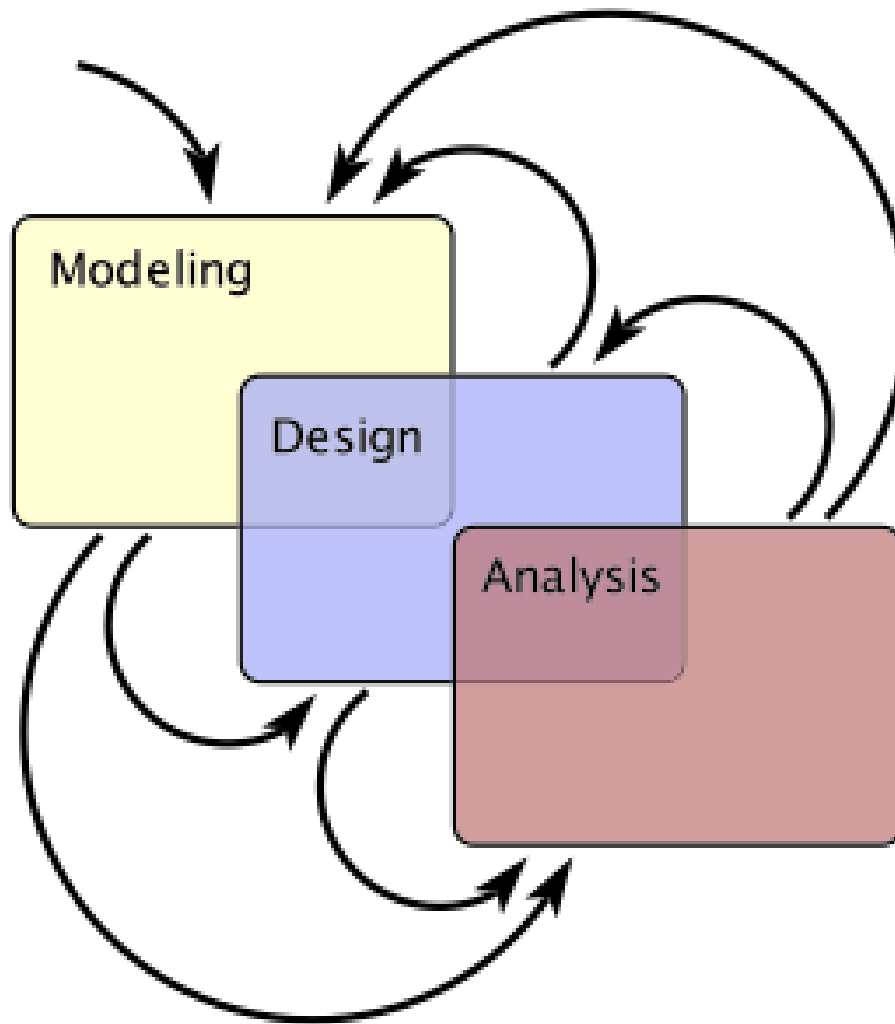
- State machines
 - sequential decision logic
- Synchronous/reactive concurrent composition
 - concurrent computation
 - composes well with state machines
- Dataflow models
 - exploitable parallelism
 - well suited to signal processing
- Discrete-event models
 - explicit about time
- Time-driven
 - suitable for periodic, timed actions
- Continuous-time models
 - models of physical dynamics
 - extended to “hybrid systems” to embrace computation

Key Concepts in Model-Based Design

- Models describe physical dynamics.
- Specifications are executable models.
- Models are composed to form designs.
- Models evolve during design.
- Deployed code may be (partially) generated from models.
- Modeling languages have semantics.
- Modeling languages themselves may be modeled (meta models)
- For embedded systems, this is about
 - Time
 - Concurrency
 - Dynamics

The Design Process

- Modeling
- Design
- analysis



The Design Process(cont's)

- Modeling
 - What a system does
- Design
 - How a system does what it does
- Analysis
 - A deeper understanding of a system.
 - Why a system does what it does

modeling

- Modeling of physical dynamics, focusing on continuous dynamics in time .—chap2
- models of discrete dynamic behavior. using state machines as the principal formalism ---chap3
- combines the two with a discussion of hybrid systems ---chap4
- concurrent composition of state machines, ---chap5
- an overview of concurrent models of computation, including many of those used in design tools that practitioners frequently leverage, such as Simulink ---chap6

design

- Chapter 7 (Embedded Processors)
- Chapter 8 (Memory Architectures)
- Chapter 9 (Input and Output)
- Chapter 10 (Multitasking)
- Chapter 11 (Scheduling) introduces real-time scheduling,

analysis

- It is essential to certify that the system meets its requirements.
- system requirements are also called **properties** or **specifications**.

analysis

- Chapter 12 (Invariants and Temporal Logic) focuses on temporal logics,
- Chapter 13 (Equivalence and Refinement) focuses on the relationships between models.
- Chapter 14 Reachability Analysis and Model Checking
- Chapter 15 Quantitative Analysis is about analyzing quantitative properties of embedded software,

summary

- CPS: combine computation, communication, and physical dynamics
- an overview of the engineering principles addressed for modeling, designing, and analyzing CPS